实验六　myBatis 和 Spring 整合

一、实验目的及任务

通过该实验，掌握 mybatis 和 spring 整合方法，掌握生成 mapper 实现类的生成方式，掌握 spring 事务的基本注解实现方法。

二、实验环境及条件

主机操作系统为 Win11，Tomcat 8 ,jdk1.8 或以上版本。

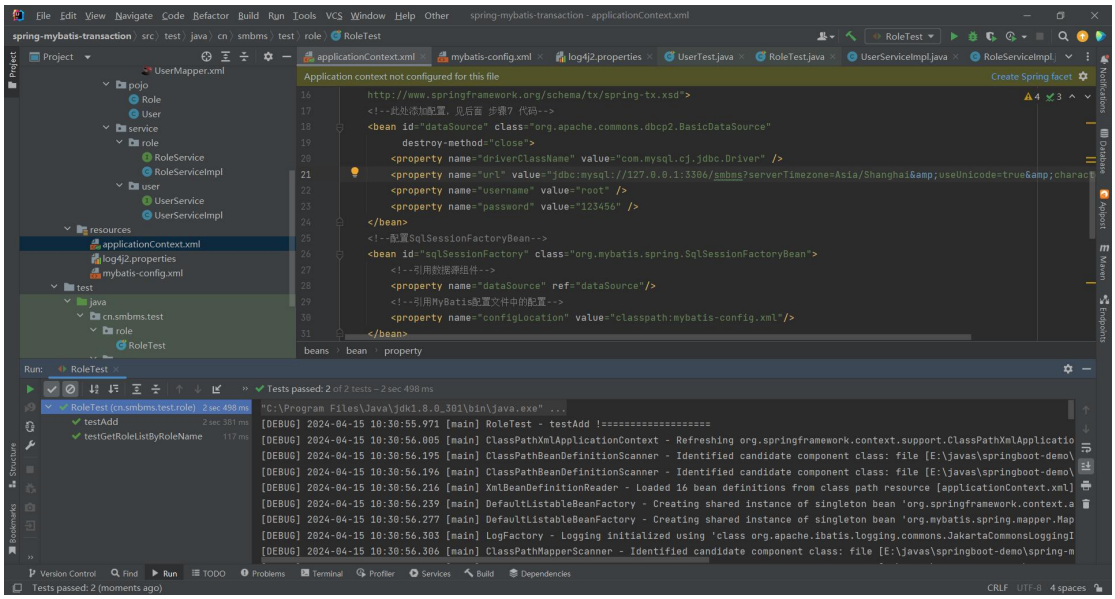
三、实验实施步骤

略

四、实验报告内容

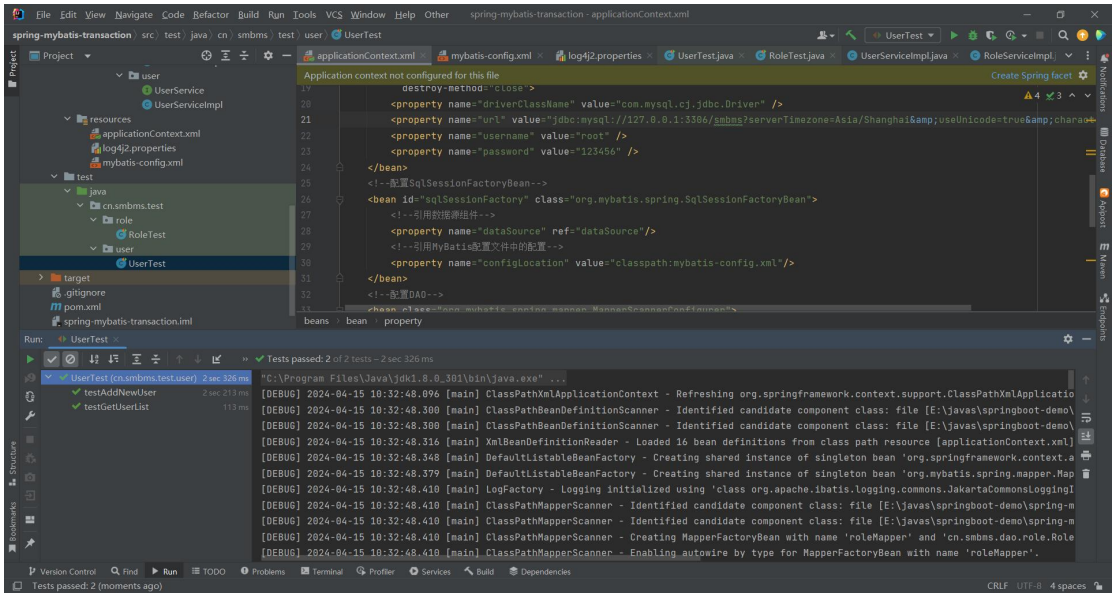
1 修改测试数据，程序在控制台的输出结果是什么？给出运行结果截图。

Roletest 测试



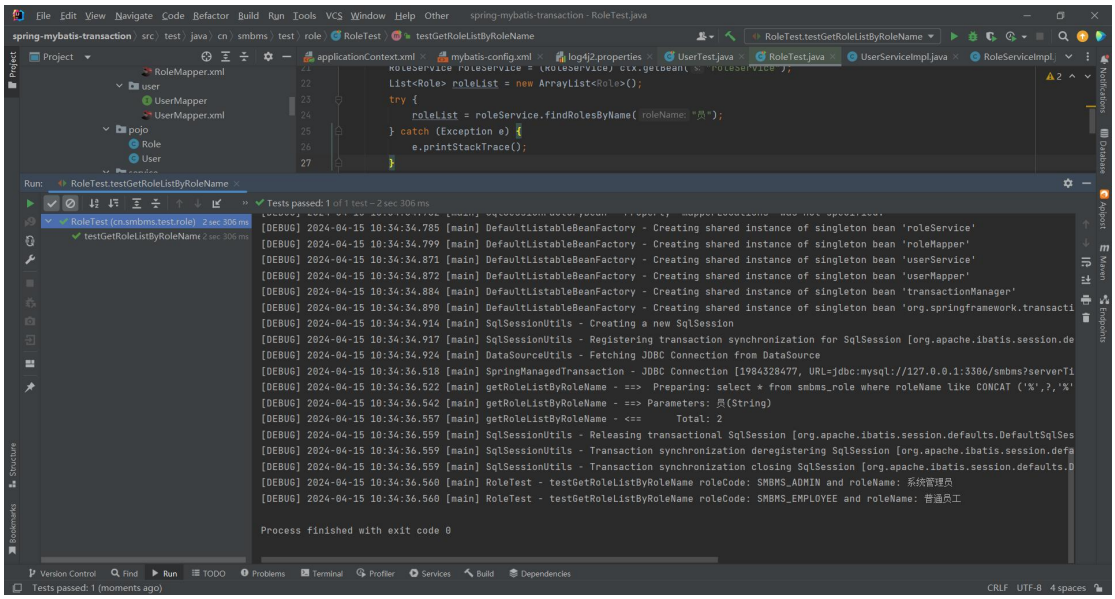
id	roleCode	roleName	createdBy	creationDate	modifyBy	modifyDate
1	SMBMS_ADMIN	系统管理员	1	2016-04-13 00:00:00	(Null)	(Null)
2	SMBMS_MANAGER	经理	1	2016-04-13 00:00:00	(Null)	(Null)
3	SMBMS_EMPLOYEE	普通员工	1	2016-04-13 00:00:00	(Null)	(Null)
4	TEST001	测试角色001	1	2024-04-15 10:30:57	(Null)	(Null)

Usertest 测试

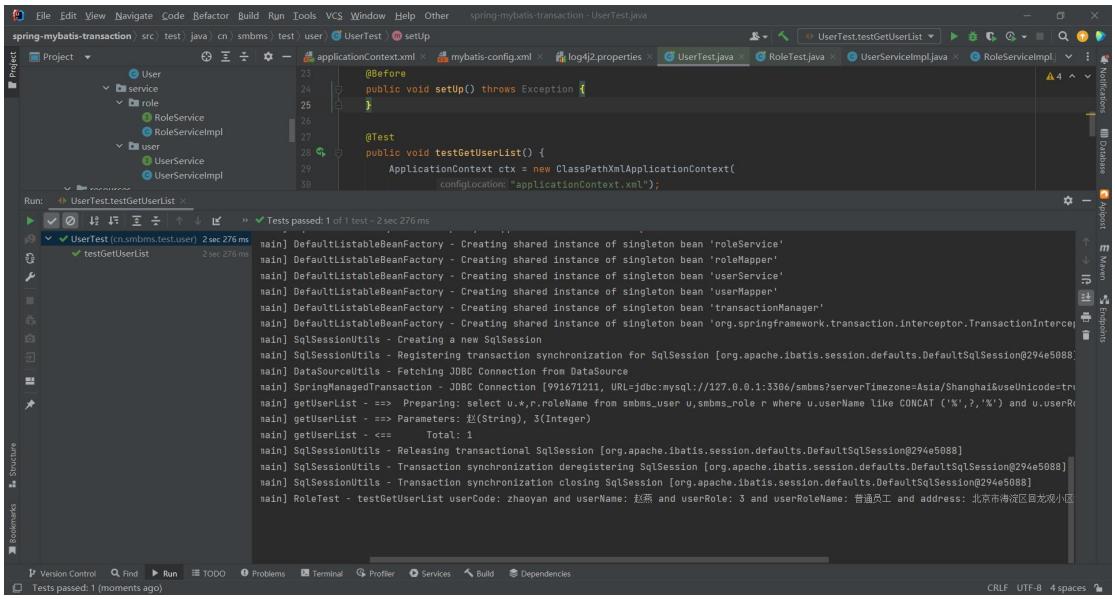


id	userCode	userName	userPassword	gender	birthday	phone	address	userRole	createdBy	creationDate	modifyBy
1	admin	系统管理员	1234567	1	1983-10-10	13688889999	北京市海淀区成府路2(	1	1	2013-03-21 16:52:07	(N
2	liming	李明	0000000	2	1983-12-10	13688884457	北京市东城区前门东大	2	1	0000-00-00 00:00:00	(N
5	hanlubiao	韩路彪	0000000	2	1984-06-05	18567542321	北京市朝阳区北辰中心	2	1	2014-12-31 19:52:09	(N
6	zhanghua	张华	0000000	1	1983-06-15	13544561111	北京市海淀区学院路6	3	1	2013-02-11 10:51:17	(N
7	wangyang	王洋	0000000	2	1982-12-31	13444561124	北京市海淀区西二旗辉	3	1	2014-06-11 19:09:07	(N
8	zhaoyan	赵燕	0000000	1	1986-03-07	18098764545	北京市海淀区回龙观小	3	1	2016-04-21 13:54:07	(N
10	sunlei	孙磊	0000000	2	1981-01-04	13387676765	北京市朝阳区管庄新月	3	1	2015-05-06 10:52:07	(N
11	sunxing	孙兴	0000000	2	1978-03-12	13367890900	北京市朝阳区建国门南	3	1	2016-11-09 16:51:17	(N
12	zhangchen	张晨	0000000	1	1986-03-28	18098765434	朝阳区管庄路口北柏林	3	1	2016-08-09 05:52:37	
13	dengchao	邓超	0000000	2	1981-11-04	13689674534	北京市海淀区北航家属	3	1	2016-07-11 08:02:47	(N
14	yangguo	杨过	0000000	2	1980-01-01	13388886623	北京市朝阳区北苑家园	3	1	2015-02-01 03:52:07	(N
15	zhaomin	赵敏	0000000	1	1987-12-04	18099897657	北京市昌平区天通苑3(	2	1	2015-09-12 12:02:12	(N
16	test002	测试用户002	1234567	1	2002-12-12	13688783697	地址测试	1	1	2024-04-15 10:32:49	(N

testGetRoleListByRoleName 测试



testGetUserList 测试



2 MapperFactoryBean 和 `MapperScannerConfigurer` 类的作用是什么？

1. `MapperFactoryBean`：
   - `MapperFactoryBean` 是一个 FactoryBean，它用于创建 MyBatis 映射器（Mapper）的代理实例。
   - 它将 Spring 的依赖注入（DI）和 MyBatis 的映射器接口集成在一起，允许你在 Spring 应用上下文中将 MyBatis 映射器作为 Bean 使用。
   - 通常，你需要在 Spring 配置文件中为每个映射器接口定义一个`<bean>`，指定 `MapperFactoryBean` 作为其 class，并通过`<property>`指定映射器接口的类型。这样，Spring 会自动创建映射器接口的代理实例，并注入到其他需要该映射器的 Bean 中。

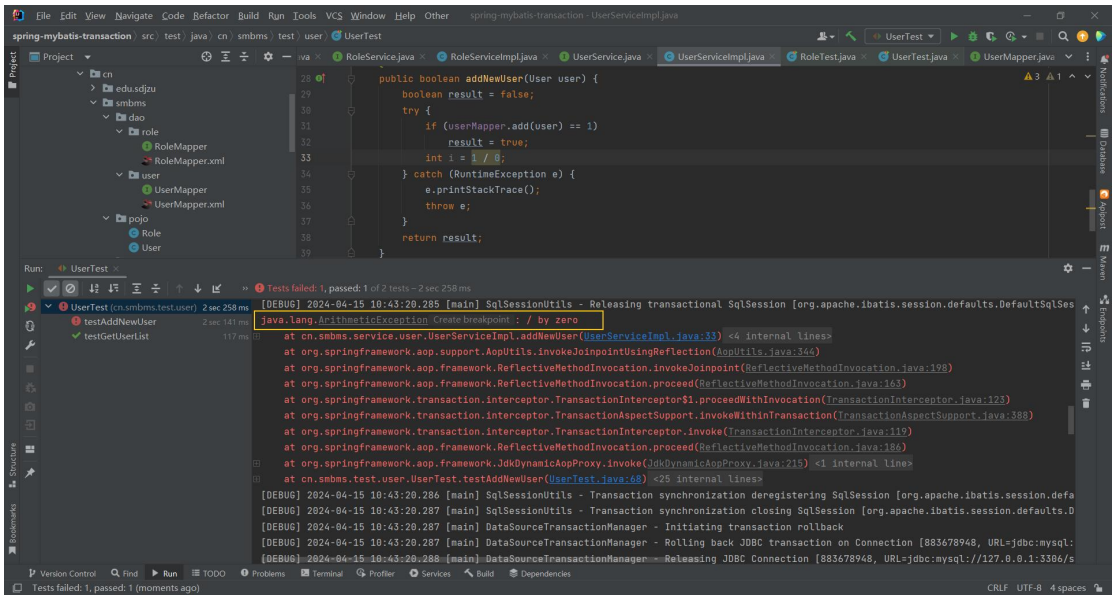
2. `MapperScannerConfigurer`:

   - `MapperScannerConfigurer` 是一个 Bean 定义的扫描器，它自动创建映射器接口的代理实例，无需为每个映射器接口单独定义 Bean。
   - 使用 `MapperScannerConfigurer` 时，你只需指定一个或多个包路径，它会扫描这些包中的所有接口，并为它们创建代理实例。
   - 这个类简化了配置，当你有很多映射器接口时，它特别有用，因为它减少了重复的配置工作。
   - `MapperScannerConfigurer` 会自动将这些代理实例注册到 Spring 应用上下文中，你可以像使用其他 Spring Bean 一样注入它们。

MapperFactoryBean 用于单个映射器接口的配置，而 MapperScannerConfigurer 用于批量扫描和配置映射器接口，两者都使得 MyBatis 映射器能够无缝集成到 Spring 框架中。

3 服务层中 `UserServiceImpl` 和 `RoleServiceImpl` 中的 `addNewUser` 和 `addNewRole` 方法中，如果将`//int i = 1 / 0;`的注释放开，即出现一个除零的语句。控制台将出现什么结果，数据库保存操作是否进行了回滚？是由哪些配置实现的？

出现 `ArithmeticException`，因为尝试除以零是非法的。



Spring 事务管理器会标记当前事务为回滚状态，导致任何已经进行的数据库操作都会被回滚，即数据库保存操作不会提交。

是由@Transactional 注解：这个注解用于声明方法或类应该在一个事务的上下文中执行。当方法执行成功时，事务被提交；如果方法抛出运行时异常，事务被回滚。还有 Spring 的事务管理器：在 Spring 配置文件中，通常会定义一个或多个事务管理器（如 DataSourceTransactionManager），它们负责创建、提交或回滚事务。最后是 Spring 的 AOP（面向切面编程）：Spring 使用 AOP 来实现声明式事务管理。当调用被 @Transactional 注解标记的方法时，Spring 会创建一个代理对象来拦截方法调用，并在方法开始时开启一个事务，在方法正常完成时提交事务，在方法抛出异常时回滚事务。

4 你遇到了哪些问题，如何解决的？

1. 数据库配置未修改为 127.0.0.1:3306 导致数据库报错。
2. 对于报错信息不了解，通过查阅分析报错，解决。
3. 通过本次实验，掌握了 mybatis 和 spring 整合方法，学会了生成 mapper 实现类的生成方式，以及 spring 事务的基本注解实现方法。